Informacijski sistem za preverjanje plagiatov v programski kodi

(Projektna naloga pri predmetu Sistemi III)

Ime in priimek: _____

Leto oddaje osnutka *(če je bil osnutek oddan v ocenjevanje):* _____

Način opravljanja predmeta *(obkroži): pisni izpit / implementacija*

Opredelitev problema

Navodila

- *Kaj je problem?*
- *Kje in zakaj se problem pojavi?*
- *Kdo problem občuti? Predstavite več perspektiv.*
- *Kaj je kontekst/obseg problema, ki ga rešujete?*

Ob definiciji problema se moraš zavedati, da je ključnega pomena, da se vsi deležniki strinjajo o problemu. To vključuje uporabnike in naročnika.

Naloga

- *Za temo po vašem izboru zapiši opredelitev problema.*

Na Fakulteti za matematiko, naravoslovje in informacijske tehnologije Univerze na Primorskem študenti pri predmetih s programerskega stebra kot del obveznosti rešujejo t. i. velike domače naloge. Pri oddaji teh nalog se pogosto srečujemo s težavo nedovoljenega prepisovanja, kjer študenti kopirajo rešitve svojih kolegov. Ročno preverjanje in iskanje podobnosti med oddanimi rešitvami je zamudno in pogosto vodi zgolj do okvirne ocene stopnje prepisovanja.

Za natančnejše odkrivanje podobnosti bi potrebovali sistem, ki omogoča primerjavo dveh dokumentov in poda stopnjo ujemanja v odstotkih. Pri programski kodi zgolj tekstovna primerjava ni dovolj, saj lahko že z zamenjavo imen spremenljivk dobimo dve skoraj identični rešitvi z zelo nizko oceno podobnosti. Z razvojem tehnologij za analizo besedil so danes na voljo orodja, ki lahko presežejo te omejitve in omogočajo semantično analizo kode.

Nedovoljeno prepisovanje predstavlja resno težavo ne le za univerzo, ki si prizadeva za kakovost in ugled svojih študijskih programov, temveč tudi za poštene študente, ki niso uporabljali tovrstnih nedovoljenih praks. Ti študenti zaradi nepravičnega ocenjevanja ne dobijo zasluženih ocen, kar lahko vpliva na njihovo pridobivanje štipendij in možnosti za zaposlitev. Posledično se lahko taka negativna selekcija odrazi tudi na širši družbeni ravni.

V okviru seminarja pri predmetu Sistemi III želimo razviti sistem, ki bo naslavljal to problematiko. Sistem bomo zasnovali tako, da ga bo mogoče vključiti v delovno okolje predavateljev in študentov na Univerzi na Primorskem ter jim tako olajšati odkrivanje in preprečevanje plagiatorstva.

Funkcijske in nefunkcijske zahteve novega sistema

Predavanje: *Sis3 - 09 [CH6] Odkrivanje zahtev (Requirements Discovery)*

Navodila
Izvedite zajem zahtev tako, da prevzamete vlogo sistemskega analitika, ki v sodelovanju z uporabniki identificira obstoječe težave in potrebe po novem informacijskem sistemu. Sistemske zahteve predstavljajo osnovo za razvoj sistema in vključujejo opise funkcionalnosti, lastnosti (atributov) ter

morebitnih omejitev, ki jih je treba upoštevati v danem kontekstu. Sistemske zahteve razvrščamo na fukcijske in nefunkcijske zahteve:

***Funkcijska zahteva** je funkcija ali funkcionalnost, ki jo je treba vključiti v informacijski sistem, da bi zadovoljili poslovne in uporabnikove potrebe. Kaj naj sistem počne?*

***Nefunkcijskih zahteva** je opis karakteristik in lastnosti sistema ter morebitne omejitve, ki lahko postavljajo meje predlaganim rešitvam. Kako naj sistem deluje?*

Nefunkcijskih zahteve

Vrsta zahteve	Pojasnilo
Zmogljivost	Zahteve glede zmogljivosti, ki jih mora sistem izpolnjevati, da zadovolji potrebe uporabnikov. ● Kakšna je sprejemljiva hitrost pretoka? ● Kakšen je sprejemljiv odzivni čas?
Informacije	Zahteve po informacijah predstavljajo informacije, ki glede vsebine, pravočasnosti, natančnosti in oblike ustrezajo uporabnikom. ● Kako ažurne morajo biti informacije? ● Kakšni so vmesniki do zunanjih sistemov?
Ekonomičnost	Ekonomične zahteve predstavljajo potrebo po sistemu za zmanjšanje stroškov in povečanje dobička. ● Katera so področja sistema, kjer je treba zmanjšati stroške? ● Za koliko bi morali zmanjšati stroške ali povečati dobiček? ● Kakšne so proračunske omejitve? ● Kakšen je časovni načrt za razvoj?
Nadzor (in varnost)	Zahteve za nadzor predstavlja okolje, v katerem mora sistem delovati, ter vrsto in stopnjo varnosti, ki jih je treba zagotoviti. ● Ali je potrebno nadzorovati dostop do sistema ali informacij? ● Katere so zahteve glede zasebnosti? ● Ali kritičnost podatkov zahteva potrebo po posebnem ravnanju (varnostne kopije, shranjevanje zunaj kraja itd.) s podatki?
Učinkovitost	Zahteve po učinkovitosti predstavljajo zmožnost sistemov za proizvodnje z minimalnimi odpadki. ● Ali v postopku obstajajo podvojeni koraki, ki jih je treba odpraviti? ● Ali obstajajo načini za zmanjšanje odpadkov pri uporabi virov?
Stortive	Zahteve storitev predstavljajo potrebe o zanesljivosti, prilagodljivosti in razširljivosti sistema. ● Kdo bo uporabljal sistem in kje se bo nahajal? ● Ali bodo obstajali različni tipi uporabnikov? ● Katere naprave za usposabljanje in gradivo za usposabljanje je treba vključiti v sistem? ● Kakšne so zahteve glede zanesljivosti / razpoložljivosti? ● Kako naj se sistem distribuira? ● Kakšna dokumentacija je potrebna?

[Vir] Poglavje 6: Fact-Finding Techniques for Requirements Discovery, Systems Analysis and Design Methods, Whitten & Bentley

Naloga

- *Za temo po vašem izboru zapiši Funkcijske in nefunkcijske zahteve novega sistema.*

Funkcijske zahteve sistema

Sistem mora omogočati naslednje funkcionalnosti:

1. Upravljanje predmetov in študijskih let izvajanja
    - Uporabniki lahko ustvarjajo, urejajo in brišejo predmete. Vsak predmet se lahko izvaja v enem ali več študijskih letih.
    - Sistem shranjuje osnovne podatke o vsakem predmetu, kot so naslov predmeta, število kontaktnih ur predavanj in vaj ter število kreditnih točk ECTS.
    - Za vsako študijsko leto, v katerem se predmet izvaja, sistem shranjuje podatke o izvajalcih, ki sodelujejo pri tem predmetu, njihovih vlogah in številu kontaktnih ur, dodeljenih posameznemu izvajalcu. Izvajalec ima lahko pri različnih predmetih različne vloge, pri istem predmetu pa lahko opravlja tudi več vlog.
    - Sistem shranjuje tudi osnovne podatke o uporabnikih, vključno z imenom, priimkom, vlogo v sistemu, datumom dodelitve vloge in obdobjem veljavnosti vloge.

2. Upravljanje nalog in oddaj

    - Sistem uporabnikom omogoča oddajanje rešitev nalog, vendar mora biti vsaka naloga najprej ustvarjena in povezana z določenim študijskim letom.
    - Ob ustvarjanju naloge morajo biti navedeni naslednji podatki: predmet, kateremu naloga pripada, študijsko leto, ime naloge, opis naloge in izvorna datoteka.
    - Ko je naloga ustvarjena, lahko uporabniki začnejo oddajati svoje rešitve. Sistem uporabnikom omogoča brisanje posameznih oddaj ali celotnih nalog, če je to potrebno. Vse datoteke so shranjene anonimno, zabeleži pa se tudi datum oddaje.

3. Obdelava rešitev in zgodovina obdelav

    - Uporabniki lahko hkrati pošljejo eno ali več oddaj v primerjavo.
    - Sistem shranjuje podatke o tem, kdo je zahteval primerjavo in katere oddaje so bile primerjane. Vsaka uspešno izvedena primerjava se zabeleži v sistem, uporabniki pa lahko dostopajo do zgodovine primerjav.
    - Za vsako primerjavo sistem shranjuje čas obdelave, datum izvedbe, uporabnika, ki je sprožil primerjavo, ter rezultate primerjave v besedilni obliki in v obliki ocene podobnosti.
    - Primerjavo lahko sprožijo samo izvajalci predmetov (npr. predavatelji ali asistenti).

4. Vizualizacija rezultatov

Sistem omogoča dva načina prikaza rezultatov primerjave:

- Tabelarični pogled: tabela, ki prikazuje odstotek podobnosti med oddajami.
- Grafični pogled: utežen graf, kjer vozlišča predstavljajo posamezne oddaje, povezave med njimi pa stopnjo podobnosti. Povezava se prikaže le, če podobnost presega 40 %.

5. Uporabniške vloge in dostop

- Dostop do sistema in njegovo uporabo imajo samo registrirani uporabniki.
- Vsakemu uporabniku je dodeljena vloga z določenim obdobjem veljavnosti.
- Vsak predmet ima nosilca, ki lahko spreminja karkoli glede predmeta, kot so dodelitve izvajalcev, število kreditnih točk ECTS in kontaktne ure.
- Superuporabnik ima popoln dostop do vseh vsebin v sistemu.

Nefunkcijske zahteve sistema

1. Sistem mora biti sposoben obdelati do 5000 oddaj na minuto.
2. Sistem bi moral zagotoviti rezultate primerjave vsak-z-vsakim za do 500 oddaj v 5 minutah ali manj.
3. Sistem mora omogočati integracijo v platformo MOODLE.
4. Sistem je potrebno razviti in vpeljati v treh mesecih od zaključka zasnove sistema.
5. Podatke je treba varno hraniti najmanj 12 mesecev. Podatke je treba hraniti na najmanj dveh nosilcih podatkov. Zunanje varnostne kopije je potrebno izvajati na tedenski osnovi.
6. Sistem bodo uporabljali študenti in učitelji, ki bi morali imeti dostop do sistema s poljubne lokacije z osebnim računalnikom, ki je povezan z internetom.
7. Sistem mora podpirati dva tipa uporabnikov: učitelji in študenti.
8. Sistem upravlja z osebnimi podatki občutljive narave zato mora zagotavljati varno komunikacijo med uporabniki in sistemom. Izmenjava podatkov mora vedno potekati po kriptirani povezavi.
9. Sistem mora zagotvaljati 24/7 delovanje s procentom časa aktivnega delovanja (up-time) minimalno 95% .
10. Sistem naj bo namestljiv kot spletna aplikacija. Namestitev in konfiguracijo je potrebno omogočiti preko avtomatiziranih namestitvenih skript.
11. Sistem mora vključevati dokumentacijo za namestitev, vzdrževanje in integracijo sistema.

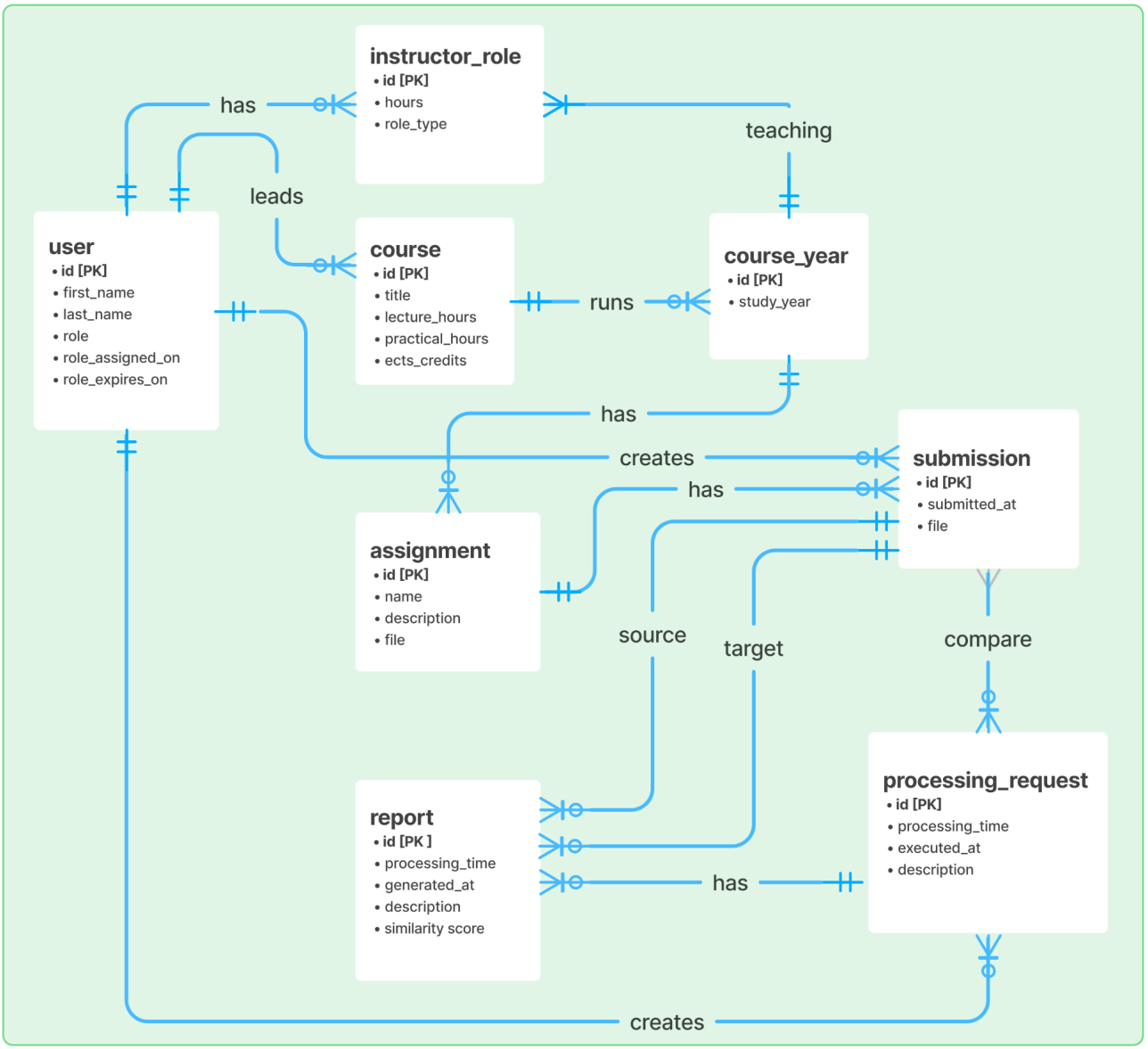
Logično Načrtovanje

Podatkovno modeliranje

Entitetno relacijski diagram (ER diagram)

Naloga:

- *Za izbrano temo izdelajte diagram ER. Upoštevajte navodila. V pomoč vam je Poglavje 8: Podatkovno modeliranje in analiza (Chapter 8: Data Modeling and Analysis).*
- *Za risanje je potrebno uporabiti brezplačno aplikacijo https://www.figma.com/*
- *V seminarju morate oddati tudi povezavo do izdelanega grafa na platformi figma.*
- *Prepričajte se, da znate izdelati ER-diagrame, saj boste to morda morali pokazati na ustnem zagovoru.*



Slika 1: Entitetno-relacijski diagram sistema za zaznavanje plagiatorstva izvorne kode. Diagram je na voljo na povezavi.

Podatkovni slovar

Naloga:

- *Za izbrano temo izdelajte diagram ER. V pomoč vam je Poglavje 8: Podatkovno modeliranje in analiza (Chapter 8: Data Modeling and Analysis).*

Table 1: Data dictionary for a source code plagiarism detection system

Entity Name	Entity Desc.	Entity Attribute	Attribute Desc.	Attribute Domain	Attribute Def. Val.	Attribute Constraint
user	Stores information about registered users of the system.	id	Unique identifier of the user.	INTEGER	auto-generated	Primary key, not null, unique
		first_name	User's given name.	VARCHAR(100)	null	Not null
		last_name	User's family name.	VARCHAR(100)	null	Not null
		role	User's role in the system.	ENUM("student", "employee", "admin")	'student'	Not null
		role_assigned_on	Date when the role was assigned.	DATE	CURRENT_DATE	Not null
		role_expires_on	Date when the role expires.	DATE	null	Must be greater than or equal to role_assigned_on
Entity Name	**Entity Desc.**	**Entity Attribute**	**Attribute Desc.**	**Attribute Domain**	**Attribute Def. Val.**	**Attribute Constraint**
course	Stores basic information about a course.	id	Unique identifier of the course.	INTEGER	auto-generated	Primary key, not null, unique
		title	Official title of the course.	VARCHAR(200)	null	Not null
		lecture_hours	Number of lecture hours.	INTEGER	0	Must be greater than or equal to 0
		practical_hours	Number of practical hours.	INTEGER	0	Must be greater than or equal to 0
		ects_credits	Number of ECTS credits.	INTEGER	0	Must be greater than or equal to 0
Entity Name	**Entity Desc.**	**Entity Attribute**	**Attribute Desc.**	**Attribute Domain**	**Attribute Def. Val.**	**Attribute Constraint**
course_year	Stores a course in a specific academic year.	id	Unique identifier of the course offering.	INTEGER	auto-generated	Primary key, not null, unique
		study_year	Academic year of the course.	VARCHAR(9)	null	Not null, format `YYYY/YYYY`

Entity Name	Entity Desc.	Entity Attribute	Attribute Desc.	Attribute Domain	Attribute Def. Val.	Attribute Constraint
instructor_role	Stores a teaching role in a course offering.	id	Unique identifier of the role record.	INTEGER	auto-generated	Primary key, not null, unique
		hours	Number of assigned contact hours.	INTEGER	0	Must be greater than or equal to 0
		role_type	Type of teaching role.	ENUM(“lecturer”, “assistant”, “support”)	null	Not null

Entity Name	Entity Desc.	Entity Attribute	Attribute Desc.	Attribute Domain	Attribute Def. Val.	Attribute Constraint
assignment	Stores an assignment for a course offering.	id	Unique identifier of the assignment.	INTEGER	auto-generated	Primary key, not null, unique
		name	Name of the assignment.	VARCHAR(200)	null	Not null
		description	Text description of the assignment.	TEXT	null	Nullable
		file	Reference to the assignment file.	VARCHAR(255)	null	Not null

Entity Name	Entity Desc.	Entity Attribute	Attribute Desc.	Attribute Domain	Attribute Def. Val.	Attribute Constraint
submission	Stores a submitted assignment solution.	id	Unique identifier of the submission.	INTEGER	auto-generated	Primary key, not null, unique
		submitted_at	Date and time of submission.	DATETIME	CURRENT_TIMESTAMP	Not null
		file	Reference to the submitted file.	VARCHAR(255)	null	Not null

Entity Name	Entity Desc.	Entity Attribute	Attribute Desc.	Attribute Domain	Attribute Def. Val.	Attribute Constraint
processing_request	Stores a request for submission comparison.	id	Unique identifier of the request.	INTEGER	auto-generated	Primary key, not null, unique
		processing_time	Time in seconds needed for processing.	FLOAT	0	Must be greater than or equal to 0
		executed_at	Date and time of execution.	DATETIME	CURRENT_TIMESTAMP	Not null

		description	Short description of the request.	TEXT	null	Nullable

Entity Name	Entity Desc.	Entity Attribute	Attribute Desc.	Attribute Domain	Attribute Def. Val.	Attribute Constraint
report	Stores the result of a comparison.	id	Unique identifier of the report.	INTEGER	auto-generated	Primary key, not null, unique
		generated_at	Date and time of report creation.	DATETIME	CURRENT_TIMESTAMP	Not null
		description	Text description of the result.	TEXT	null	Nullable
		similarity_score	Similarity value of the comparison.	DECIMAL(5,2)	0.00	Must be between 0 and 100

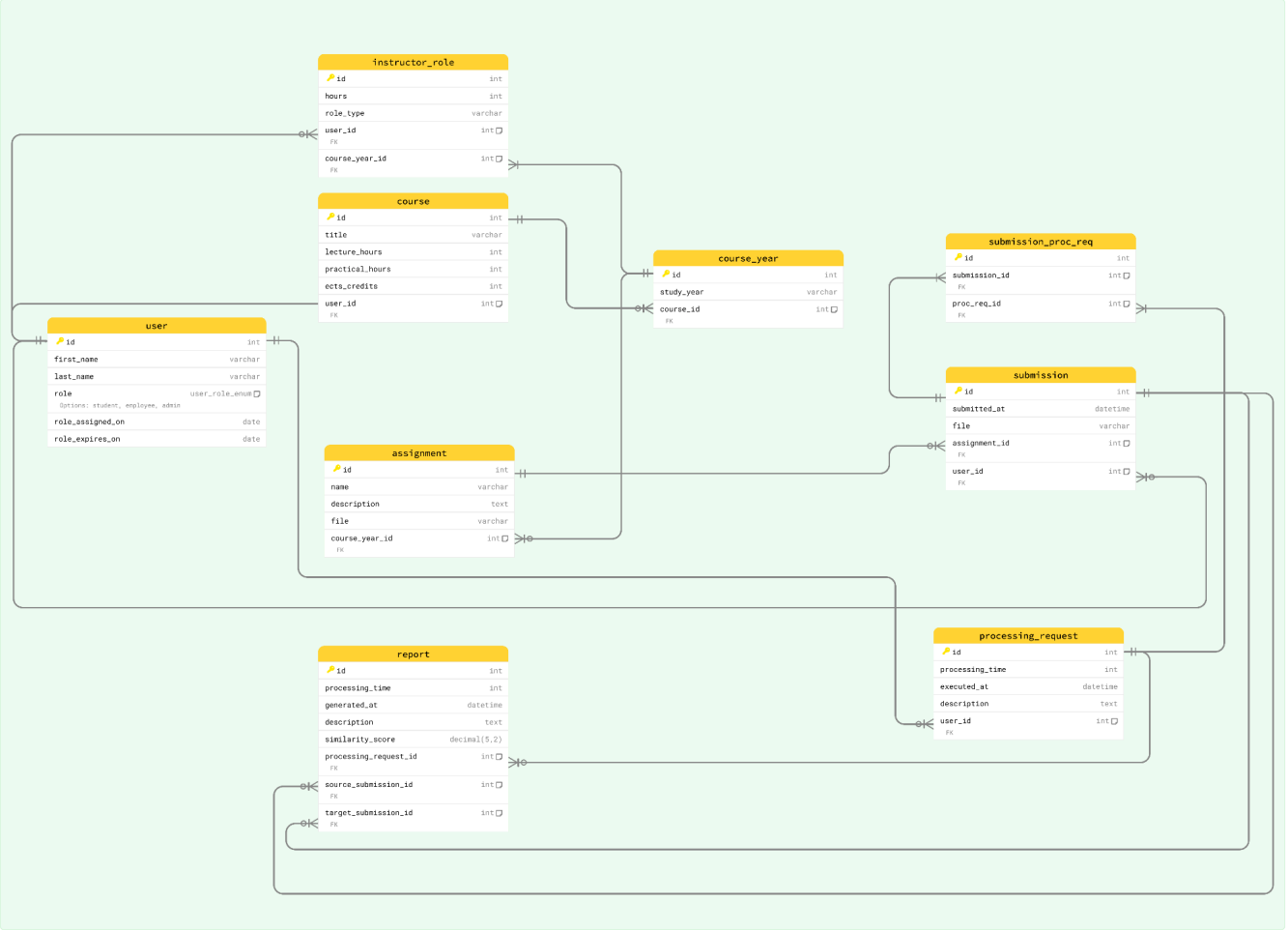
Relacijski model

Navodila:

- *Preoblikujte ER-diagram v relacijski model.*
- *Normalizirajte relacijski model do NF3.*
- *Kot vir uporabite prosojnice Sis3 - 06 - CH8-Podatkovno modeliranje in analiza podatkov.*
- *Uporabite aplikacijo https://www.figma.com/.*
- *Poleg diagrama morate oddati tudi povezavo do Figma diagrama (glejte napis pod sliko spodaj).*
- *Prepričajte se, da znate pretvoriti ER-diagram v relacijski model, saj boste to morda morali pokazati na ustnem zagovoru.*

Naloga:

- *Za temo po vašem izboru izdelajte relacijski model.*



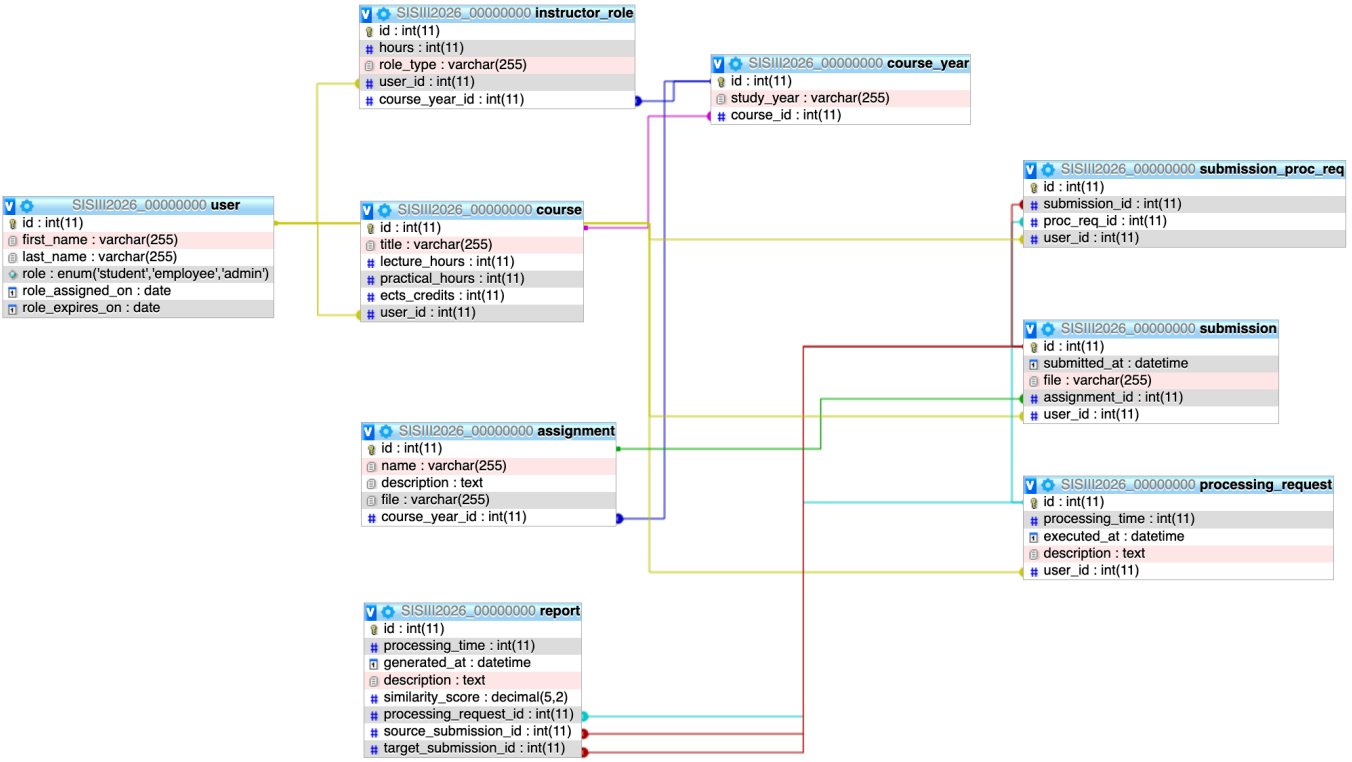
Slika 2: Relacijski model sistema za zaznavanje plagiatorstva izvorne kode. Diagram je na voljo na povezavi.

Fizično načrtovanje

Fizični podatkovni model

Naloga za danes:

- *Pretvori relacijski diagram v bazo. Definiraj vse potrebne indekse.*
- *V bazo vnesi nekaj podatkov.*
- Za dokončanje naloge uporabi mysql bazo:
    - Ime baze: SISIII2026_[vpisna_številka]
    - Collation: utf8_unicode_ci
    - uporabniško ime: studenti
    - geslo: S039C8R7
    - *http://88.200.63.148/phpmyadmin*
- Poročilo mora vključevati posnetek zaslona fizičnega modela (npr. pogled Designer v phpMyAdmin).
- Prepričajte se, da znate vnašati podatke v podatkovno bazo in spreminjati njeno strukturo, vključno z dodajanjem tabel, indeksov in omejitev tujih ključev, saj boste te naloge morda morali prikazati na ustnem zagovoru.
- Ustvarite zaslonski posnetek, ki prikazuje del postopka izdelave podatkovne baze. Posnetek mora najmanj vključevati ustvarjanje dveh tabel, dveh indeksov, ene omejitve tujega ključa in enega vnosa podatkov. V napisu pod Sliko 3 navedite javno dostopen URL-naslov, na katerem si je mogoče ogledati video.



Slika 3: Fizični podatkovni model. Video izdelave je na voljo na povezavi tukaj.

Wireframe diagrami

Naloga za danes
Vaša specifikacija mora vsebovati popoln niz wireframe diagramov za rešitev problema in navajati potek procesa prek različnih vmesnikov za vsak primer uporabe. Oddate povezavo do ene Figma datoteke kjer so razvidni vsi primeri uporabe.

Primer dobrega diagrama žičnega modela za aplikacijo Presence:

https://www.figma.com/design/FmVylqLGfFaF53RtYf1EpY/Example-Wireframe-for-Presence-App?node-id=0-1&t=wKscH133GBH6aJPW-1

Information system for checking plagiarism in the code

(Project for Systems III)

Name and surname: _____

Year of draft submission (if a draft was submitted for evaluation): _____

Method of course completion (circle): written exam / implementation

Problem statement

Instructions

- *What is the problem?*
- *Where the problem arises?*
- *Who is feeling the consequences? Present multiple perspectives.*
- *What is the context/scope for solving the problem?*

As you are defining the problem keep in mind that users and management must agree on the problem and opportunities it opens.

Task

- *For the seminar topic of your choice write the definition of the problem.*

At the Faculty of Mathematics, Natural Sciences, and Information Technologies at the University of Primorska, students enrolled in programming courses are required to complete homework assignments as part of their curriculum. However, a recurring issue during submission is unauthorized copying where students replicate the solutions of their peers. Manually checking and comparing these submissions is time-consuming and typically yields only a rough estimate of potential plagiarism.

To more accurately detect similarities, we need a system capable of comparing two documents and returning a match percentage. In the context of source code, simple textual comparison is insufficient, as merely changing variable names can create superficially different, but functionally identical submissions. With advances in text and code analysis technologies, tools now exist that go beyond surface-level comparison and can perform semantic analysis of code.

Unauthorized copying poses a serious problem not only for the university, which strives to uphold the quality and reputation of its academic programs, but also for honest students who complete their work independently. These students may be unfairly graded, which can impact their academic standing, scholarships, and future employment opportunities. This kind of adverse selection can ultimately have broader societal consequences.

In the Systems III seminar, our goal is to develop a system that effectively addresses this challenge. We aim to design a solution that can be seamlessly integrated into existing academic workflows.

Functional and nonfunctional requirements of new system

Lecture: *Sis3 - 09 [CH6] Odkrivanje zahtev (Requirements Discovery)*

Instructions:
Here you need to perform requirements discovery in which systems analysts try to identify or extract system problems and solution requirements from the user community. A system requirement (also called a business requirement) is a description of the needs and desires for an information system. A

requirement may describe functions, features (attributes), and constraints. One can define a system requirement by listing functional and nonfunctional requirements.

A functional requirement is a function or feature that must be included in an information system in order to satisfy the business need and be acceptable to the users.

A nonfunctional requirement is a description of the features, characteristics, and attributes of the system as well as any constraints that may limit the boundaries of the proposed solution.

Non-functional requirement

Requirement Type	Explanation
Performance	Performance requirements represent the performance the system is required to exhibit to meet the needs of users. ● What is the acceptable throughput rate? ● What is the acceptable response time?
Information	Information requirements represent the information that is pertinent to the users in terms of content, timeliness, accuracy, and format. ● How current must the information be? ● What are the interfaces to external systems?
Economy	Economy requirements represent the need for the system to reduce costs or increase profits. ● What are the areas of the system where costs must be reduced? ● How much should costs be reduced or profits be increased? ● What are the budgetary limits? ● What is the timetable for development?
Control (and Security)	Control requirements represent the environment in which the system must operate, as well as the type and degree of security that must be provided. ● Must access to the system or information be controlled? ● What are the privacy requirements? ● Does the criticality of the data necessitate the need for special handling (backups, offsite storage, etc.) of the data?
Efficiency	Efficiency requirements represent the systems ability to produce outputs with minimal waste. ● Are there duplicate steps in the process that must be eliminated? ● Are there ways to reduce waste in the way the system uses resources?
Service	Service requirements represent needs in order for the system to be reliable, flexible, and expandable. ● Who will use the system and where are they located? ● Will there be different types of users? ● What training devices and training materials are to be included in the system? ● What training devices and training materials are to be developed and maintained separately from the system, such as stand- alone computer based training (CBT) programs or databases? ● What are the reliability/availability requirements? ● How should the system be packaged and distributed? ● What documentation is required?

[Source] CH6: Fact-Finding Techniques for Requirements Discovery, Systems Analysis and Design Methods, Whitten & Bentley

Task

- *For the seminar topic of your write functional and nonfunctional requirements.*

Functional requirements

The system should enable the following functionalities:

1. Manage Courses and Academic Years of Implementation
    - Users can create, edit, and delete courses. Each course can run in one or more academic years. The system stores basic information about each course, such as the course title, the number of lecture and practical contact hours, and the number of ECTS credits.
    - For each academic year in which the course runs, the system stores information about the instructors involved in that course, their roles, and the contact hours assigned to each instructor. An instructor can have different roles in different courses and may also have multiple roles within the same course.
    - The system also stores basic user information, including first name, last name, role within the system, the date the role was assigned, and the validity period of the role.

2. Managing Assignments and Submissions

    - The system enables users to upload assignment solutions, but each assignment must first be created and linked to a specific academic year.
    - When creating an assignment, the following information must be included: course to which it belongs, academic year, assignment name, assignment description, and the source file.
    - Once an assignment is created, users can begin submitting their solutions. The system allows users to delete individual submissions or entire assignments if necessary. All files are stored anonymously, and the submission date is recorded.

3. Solution Processing and Processing History

    - Users can submit one or more submissions for comparison at the same time.
    - The system stores information about who requested the comparison and which submissions were compared. Each successful comparison is logged in the system, and users can access the comparison history.
    - For each comparison, the system stores the processing time, execution date, the user who initiated the comparison, and the results of the comparison in textual form and as a similarity score.
    - Solution processing can be triggered only by course instructors (i.e. lecturers, assistants).

4. Visualisation of Results

The system provides two ways to display the results of the processing:

    - The system provides two ways to display the comparison results:
    - Table view: a table showing the percentage similarity between submissions.
    - Graph view: a weighted graph where nodes represent individual submissions and links represent the degree of similarity between them. A link is displayed only if the similarity exceeds 40%.

5. User Roles and Access

- Only registered users can access and use the system.
- Each user is assigned a role with a defined validity period. Each course has a leader who is allowed to modify any aspect of the course, such as instructor assignments, ECTS credits, and contact hours. A superuser has full access to all content in the system.
- This version stays simple, but reads much more clearly.
- The main thing I would still change in your original is replacing task with assignment or submission, because that inconsistency stands out the most.

Non-functional requirements

1. The system must be able to process up to 5000 submissions per minute
2. The system should provide pairwise comparison results for up to 500 submissions in max 5 minutes.
3. The system must integrate with MOODLE e-learning platform.
4. The system should be developed and deployed within 3 months of system designs being finalized.
5. The system manages personal information of a sensitive nature and therefore must ensure safe communication between users, the plagiarism detection system and the MOSS system. Data exchange must always be encrypted.
6. The data needs to be stored in a secure way for a minimum of 12 months. The data should be stored on a minimum of two data carriers at all times. External backups should be made on weekly bases.
7. The system will be used by students and educators who should be able to access the system from an arbitrary location using a personal computer connected to the internet.
8. The system should support two different types of users: educators and students.
9. The system must ensure 24/7 operation with an uptime of at least 95%.
10. The system should be packaged as a web application. The installation and configuration should be enabled by running various install scripts.
11. The system should include documentation for installation, maintenance and integration of the system.

Logical Design

Data modelling

Entity relationship diagram (ERD)

Task:

- *For the topic of your choice, make ER diagram. Follow instructions from slides: Poglavje 8: Podatkovno modeliranje in analiza (Chapter 8: Data Modeling and Analysis).*
- *Use this application https://www.figma.com/*
- *Besides the diagram you also need to submit a link to figma diagram (see figure caption below).*
- *Make sure you know how to create ER diagrams as as you might get asked to do so on oral defence.*

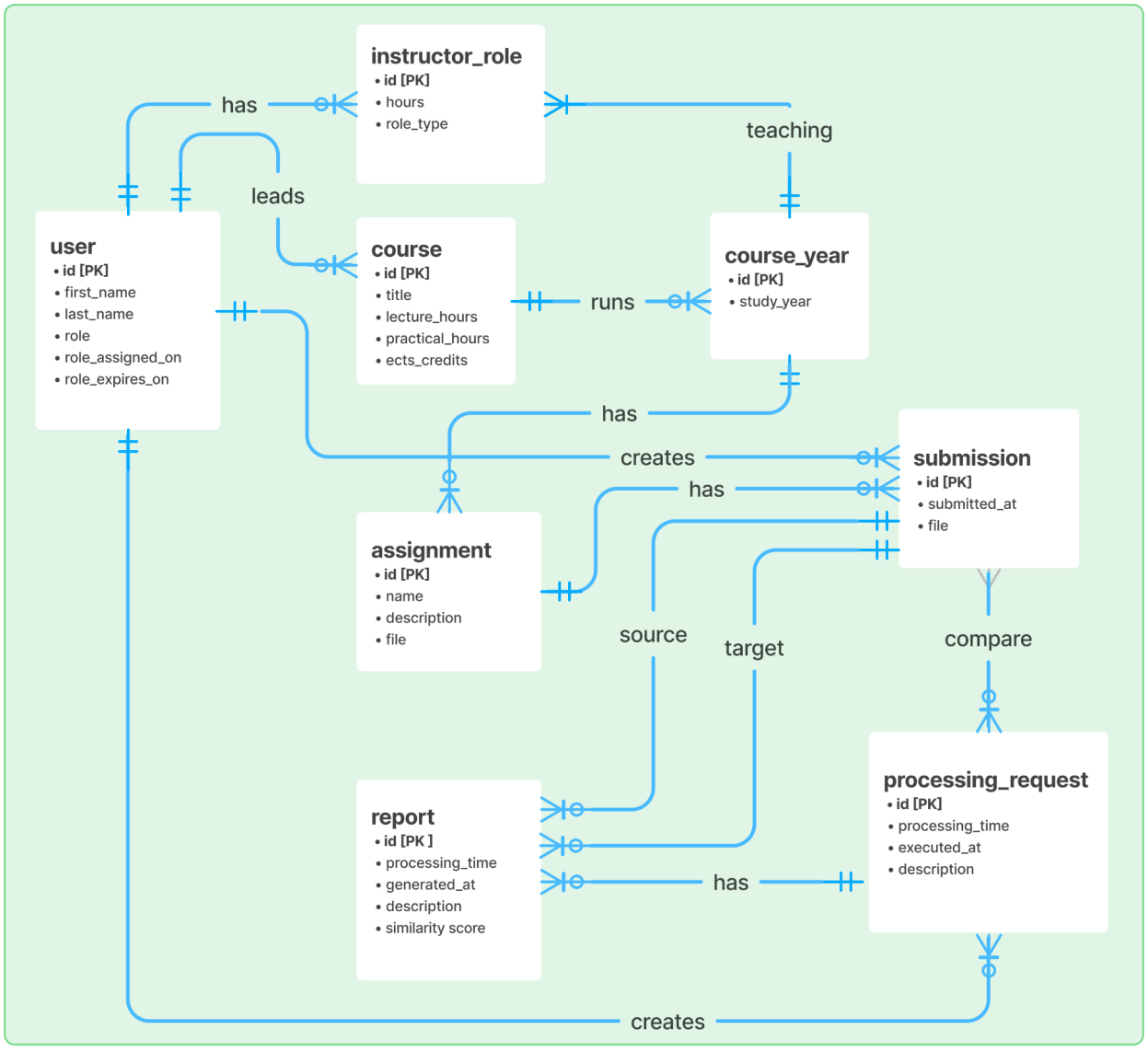


Figure 1: Entity-relationship diagram of a source code plagiarism detection system. The diagram is available at link.

Data dictionary

Task

For the seminar topic of your choice, create a data dictionary.

Table 1: Data dictionary for a source code plagiarism detection system

Entity Name	Entity Desc.	Entity Attribute	Attribute Desc.	Attribute Domain	Attribute Def. Val.	Attribute Constraint
user	Stores information about registered users of the system.	id	Unique identifier of the user.	INTEGER	auto-generated	Primary key, not null, unique
		first_name	User's given name.	VARCHAR(100)	null	Not null
		last_name	User's family name.	VARCHAR(100)	null	Not null
		role	User's role in the system.	ENUM("student", "employee", "admin")	'student'	Not null
		role_assigned_on	Date when the role was assigned.	DATE	CURRENT_DATE	Not null
		role_expires_on	Date when the role expires.	DATE	null	Must be greater than or equal to role_assigned_on
Entity Name	**Entity Desc.**	**Entity Attribute**	**Attribute Desc.**	**Attribute Domain**	**Attribute Def. Val.**	**Attribute Constraint**
course	Stores basic information about a course.	id	Unique identifier of the course.	INTEGER	auto-generated	Primary key, not null, unique
		title	Official title of the course.	VARCHAR(200)	null	Not null
		lecture_hours	Number of lecture hours.	INTEGER	0	Must be greater than or equal to 0
		practical_hours	Number of practical hours.	INTEGER	0	Must be greater than or equal to 0
		ects_credits	Number of ECTS credits.	INTEGER	0	Must be greater than or equal to 0
Entity Name	**Entity Desc.**	**Entity Attribute**	**Attribute Desc.**	**Attribute Domain**	**Attribute Def. Val.**	**Attribute Constraint**
course_year	Stores a course in a specific academic year.	id	Unique identifier of the course offering.	INTEGER	auto-generated	Primary key, not null, unique
		study_year	Academic year of the course.	VARCHAR(9)	null	Not null, format YYYY/YYYY

Entity Name	Entity Desc.	Entity Attribute	Attribute Desc.	Attribute Domain	Attribute Def. Val.	Attribute Constraint
instructor_role	Stores a teaching role in a course offering.	id	Unique identifier of the role record.	INTEGER	auto-generated	Primary key, not null, unique
		hours	Number of assigned contact hours.	INTEGER	0	Must be greater than or equal to 0
		role_type	Type of teaching role.	ENUM(“lecturer”, “assistant”, “support”)	null	Not null

Entity Name	Entity Desc.	Entity Attribute	Attribute Desc.	Attribute Domain	Attribute Def. Val.	Attribute Constraint
assignment	Stores an assignment for a course offering.	id	Unique identifier of the assignment.	INTEGER	auto-generated	Primary key, not null, unique
		name	Name of the assignment.	VARCHAR(200)	null	Not null
		description	Text description of the assignment.	TEXT	null	Nullable
		file	Reference to the assignment file.	VARCHAR(255)	null	Not null

Entity Name	Entity Desc.	Entity Attribute	Attribute Desc.	Attribute Domain	Attribute Def. Val.	Attribute Constraint
submission	Stores a submitted assignment solution.	id	Unique identifier of the submission.	INTEGER	auto-generated	Primary key, not null, unique
		submitted_at	Date and time of submission.	DATETIME	CURRENT_TIMESTAMP	Not null
		file	Reference to the submitted file.	VARCHAR(255)	null	Not null

Entity Name	Entity Desc.	Entity Attribute	Attribute Desc.	Attribute Domain	Attribute Def. Val.	Attribute Constraint
processing_request	Stores a request for submission comparison.	id	Unique identifier of the request.	INTEGER	auto-generated	Primary key, not null, unique
		processing_time	Time in seconds needed for processing.	FLOAT	0	Must be greater than or equal to 0
		executed_at	Date and time of execution.	DATETIME	CURRENT_TIMESTAMP	Not null
		description	Short description of the request.	TEXT	null	Nullable

Entity Name	Entity Desc.	Entity Attribute	Attribute Desc.	Attribute Domain	Attribute Def. Val.	Attribute Constraint

report	Stores the result of a comparison.	id	Unique identifier of the report.	INTEGER	auto-generated	Primary key, not null, unique
		generated_at	Date and time of report creation.	DATETIME	CURRENT_TIMESTAMP	Not null
		description	Text description of the result.	TEXT	null	Nullable
		similarity_score	Similarity value of the comparison.	DECIMAL(5,2)	0.00	Must be between 0 and 100

Relational model

Instructions:

- *Transform ER diagram into a relational model.*
- *Normalize relational model to 3NF. If you leave it in 2NF write a justification why you made this decision.*
- *As a reference use slides Sis3 - 06 - CH8-Data Modeling and Analysis.*
- *Use this application https://www.figma.com/*
- *Besides the diagram you also need to submit a link to figma diagram (see figure caption below).*
- *Make sure you know how to do the conversion from ER diagram to RM as as you might get asked to do so on oral defence.*

Task:

- *For the seminar topic of your choice, create a relational model.*

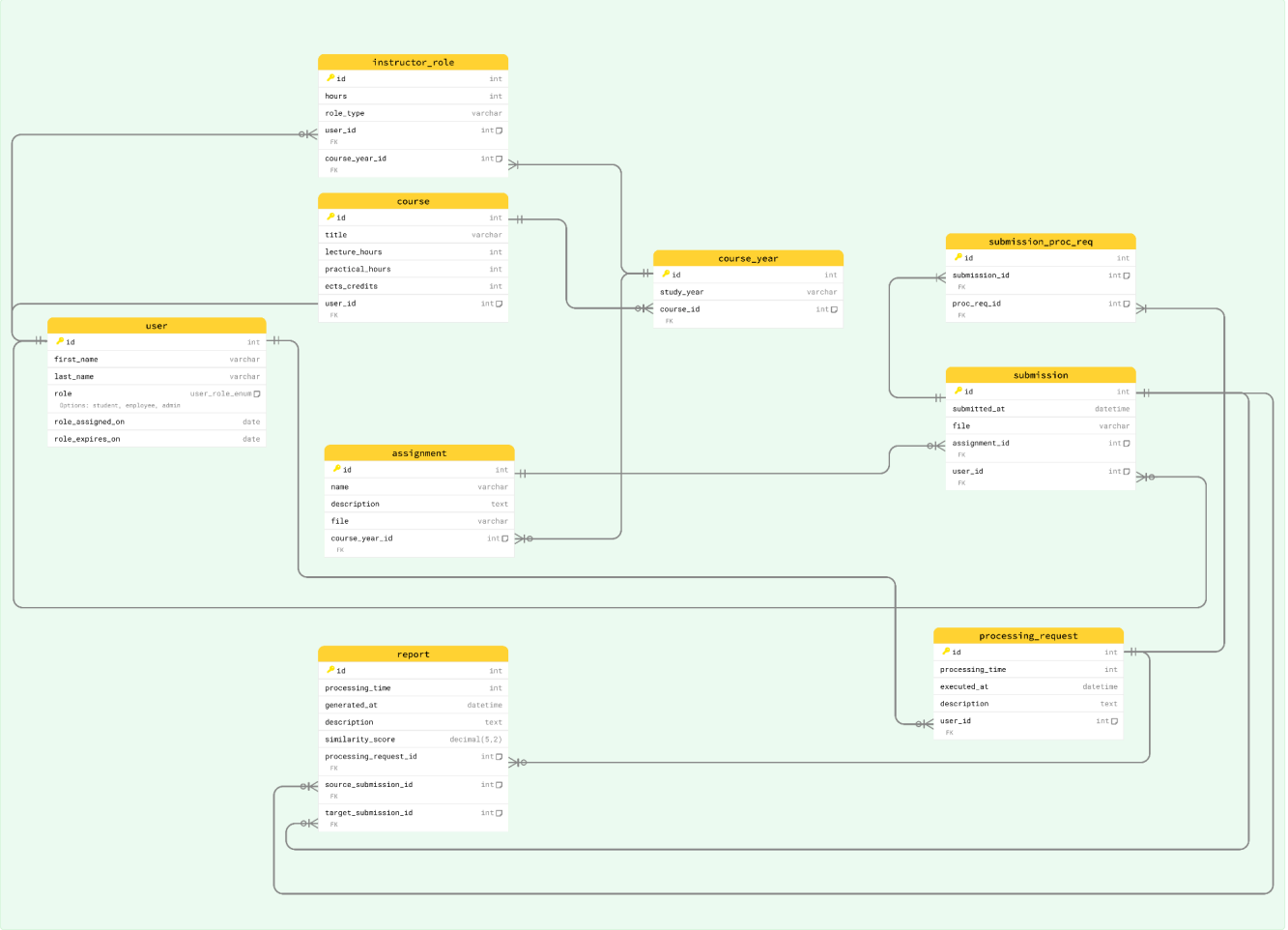


Figure 2: Relational model of a source code plagiarism detection system. The diagram is available at [link](#).

Physical Design Phase

Physical Data Model

Instructions / task for today:

- *Convert your relation model to a physical database. Define all indexes appropriately.*
- *Feed some data into your system.*
- *To complete your task use the following database:*
  - *database name: SISIII2026_ [enrollment_number]*
  - Collation: utf8_unicode_ci
  - *username: studenti*
  - *password: S039C8R7*
  - *http://88.200.63.148/phpmyadmin*
- *The report should include screenshot of the physical model (e.g. Designer view of pypmyadmin).*

- *Make sure you are able to insert data into the database and modify its structure, including adding tables, indexes, and foreign key constraints, as you may be required to demonstrate these task.*
- *Create a screen recording that shows part of the database creation process. The recording must at minimum include the creation of two tables, two indexes, one foreign key constraint and one data insert. Within the caption of Figure 3 provide a publicly accessible url address where the video can be viewed.*

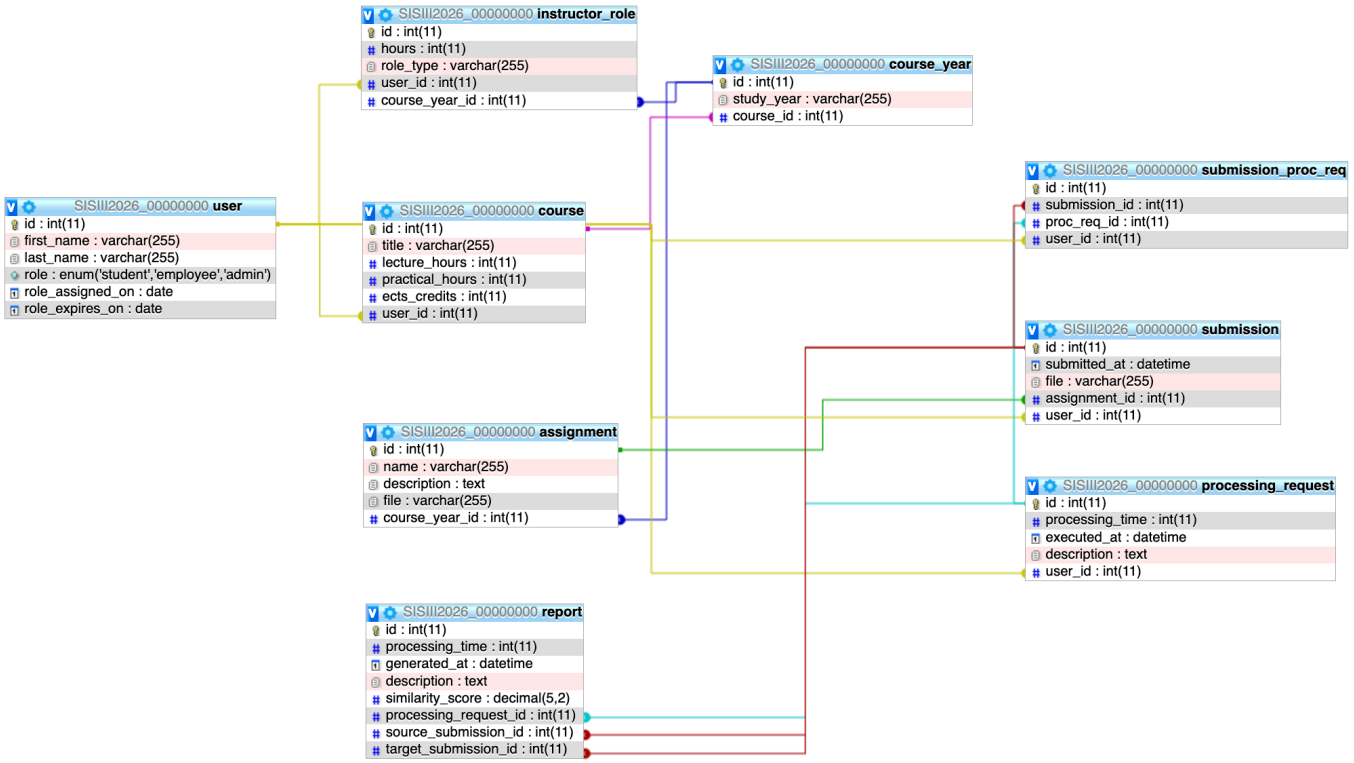


Figure 3: Physical data model. Creation video is available here.

Wireframe diagrams

Task

- Your specification must contain a complete set of wireframes for the solution, and indicate the process flow across the various interfaces for each of the use cases.

Example of a good wireframe diagram for Presence App:

https://www.figma.com/design/FmVylqLGfFaF53RtYf1EpY/Example-Wireframe-for-Presence-App?node-id=0-1&t=wKscH133GBH6aJPW-1